# SPRINTING TOWARD QUANTUM-SAFE NETWORKS

## An Incremental Scrum and AI-Assisted Engineering Process for Building and Verifying a Hybrid QKD + PQC Simulation

How increments, sprints, Definition of Done, and test oracles turn a post-quantum research idea into verifiable working software

**Truong Phat Tu • Computer Networks — <<FILL section>> • Fall 2026 • Faculty Sponsor: Dr. Victor Govindaswamy • <<FILL institution>>**

#WGP • FROM LEARNING TO BUILDING

## 01 PROBLEM & FOUNDATIONS

### THE ENGINEERING PROBLEM

Research simulations can fail silently: a program may produce plausible keys while modeling behavior incorrectly.

Changing standards and toolchains, solo development, missing tests, and unverified AI-generated material compound the risk.

Research question: What smallest disciplined process produces a running, reproducible simulation whose verified and assumed behaviors are clearly distinguished?

### CORE IDEA

**Build a working system in additive increments. Assert against theory—not yesterday's output. Keep AI outside the repository until independently checked.**

### PROCESS MODELS: FIT FOR PURPOSE

- Waterfall / plan-driven: useful for fixed external contracts and security requirements.
- Incremental: each step adds a runnable capability to the existing system.
- Integration & configuration: reuse maintained simulation and cryptography components.
- Agile / Scrum: short feedback cycles adapt to tool changes and discoveries.
- Hybrid process: standards boundary planned; simulation core incremental; toolchain integration-based.

### SCRUM — SOLO ADAPTATION

One student holds all three accountabilities. This is not full-fidelity team Scrum: CI and sponsor review are partial safeguards, not substitutes for independent challenge.

### KEY DISTINCTION

VERIFICATION — Are we building the product right? Unit, property, static, and integration tests.

VALIDATION — Are we building the right model? Compare behavior with analytical bounds and protocol evidence.

Passing tests does not prove that a simulation matches a physical link.

### BACKGROUND / DEFINITIONS

Increment: usable, additive, fully verified stepping stone that meets the Definition of Done.

Test oracle: mechanism that decides whether a stochastic output is correct.

Post-QKD here: QKD key material on selected links + PQC as scalable authentication and independent key-agreement input.

## 02 ENGINEERING PLAN

### PROCESS → NETWORK SIMULATION

INCREMENTAL DEVELOPMENT | BB84 ideal link → loss/QBER → reconciliation & privacy amplification → key API → hybrid key schedule.

TIME-BOXED SPRINTS | Two-week iterations; write the Sprint Goal before coding.

DEFINITION OF DONE | A release gate, not a feeling of completion.

INTEGRATION | Select one simulator; reuse a maintained PQC library (e.g., liboqs).

NETWORKING TOUCHPOINTS | Throughput (secret-key rate), packet loss/error control (photon loss/QBER), queuing (key pool), protocols/encapsulation (REST), security assertions.

### INCREMENT LADDER • EACH STAGE SHOULD RUN

| | | |
|---|---|---|
| I1 | **Toolchain loop** | BB84 ideal channel; sifted key; CI green |
| I2 | **Physical channel** | Loss + noise; key rate and QBER plotted |
| I3 | **Observe security** | Intercept-resend; measurable QBER rise |
| I4 | **Usable secret** | Reconciliation + privacy amplification |
| I5 | **Network service** | ETSI-style REST; key_ID, status, pool behavior |
| I6 | **Post-QKD** | liboqs ML-KEM; hybrid session key from both sources |
| I7 | **Defensible results** | Seeds, pinned versions, tagged figures, test report, AI log |

### DEFINITION OF DONE — CHECK EVERY LINE

- ☐ Runs end-to-end from one documented command; README enables reproduction.
- ☐ Unit tests for new public functions; property/metamorphic tests for probabilistic behavior.
- ☐ Analytical predictions asserted from theory; full suite passes on clean CI checkout.
- ☐ Explicit RNG seed; pinned dependencies; figures tagged with commit hash.
- ☐ AI-assisted work disclosed and verified; factual documentation cites primary sources.

### SELECTED TOOLCHAIN

Simulator: <<FILL name + pinned version>>
PQC library: <<FILL version>> • Test stack: pytest / Hypothesis / CI
Rationale: <<FILL choice based on research question>>

## 03 SYSTEM & PROCESS ARCHITECTURE

### INSERT ORIGINAL PROCESS / SYSTEM DIAGRAM HERE

Recommended visual: redraw the workflow below in draw.io / Figma / Visio

| | |
|---|---|
| **PRODUCT BACKLOG** | Product Goal • P1–P12 |
| **SPRINT BACKLOG** | Sprint Goal • WHY / WHAT / HOW |
| **BUILD LOOP** | Quantum channel → post-processing → key pool → REST API → hybrid key schedule |
| **VERIFY LOOP** | Static → unit → property → metamorphic → analytical → integration → reproducibility |
| **GATE** | PASS: usable increment \| FAIL: return to build |

SCRUM events: planning → daily log → review/demo → retrospective → backlog adaptation

### AI ASSISTANCE PLANE — ADVISORY ONLY

Allowed: literature triage • code review • test-case brainstorming • explanation checking.

Verification gate: citations opened at DOI / issuing body; AI criticism becomes a test; test ideas require theory-backed oracles; explanations checked against primary sources.

Hard rules: no unreviewed AI output committed; no AI-generated numeric results; never cite AI; do not generate cryptographic core.

Every interaction: record date, model, task, response, verification, and rejected suggestions in AI_USAGE_LOG.md.

### HYBRID PROPERTY — FEATURED TEST (MR7)

**Change ONLY the QKD key input → session key changes.**
**Change ONLY the ML-KEM shared secret → session key changes.**

This executable invariant is the core defense-in-depth claim. It demonstrates dependence on both contributors; it does not by itself prove the complete cryptographic design secure.

## 04 EVIDENCE, TRADE-OFFS & STATUS

### TEST ORACLES: HOW WILL WE KNOW?

1 ANALYTICAL BOUNDS | Sifting ≈ 1/2 of detected events; noiseless Eve-free QBER ≈ 0; post-reconciliation keys agree bit-for-bit.

2 METAMORPHIC RELATIONS | More loss must not increase key rate; more noise must not decrease QBER; same seed + parameters reproduce identical output.

3 DIFFERENTIAL TESTING | Use maintained reference implementation and its test harness for PQC components.

Statistical tests: define sample size and tolerance; use fixed seeds. Avoid unsupported numeric QBER thresholds without stated assumptions.

### INSERT REAL SIMULATION PLOT HERE

<<YOUR DATA: key rate vs. loss / QBER vs. noise>>

Caption: <<FILL seed, configuration, commit hash, sample size>>

### SPRINT METRICS • FILL WITH YOUR DATA

Backlog items Done: <<YOUR DATA>>
Line coverage / CI first-push pass rate: <<YOUR DATA>>
Metamorphic relations / defects found: <<YOUR DATA>>
AI review issues confirmed / suggestions rejected: <<YOUR DATA>>
Process overhead: <<YOUR DATA>>

### BENEFITS

- Working capability from early sprints.
- Correctness measured with checkable oracles.
- Adaptation to changing tools and requirements.
- AI support without making it an authority.
- Reproducible evidence and transferable process.

### LIMITATIONS

- Solo Scrum lacks independent challenge.
- Process overhead consumes implementation time.
- Passing tests ≠ physical hardware validity.
- Oracles can miss a uniformly wrong model.
- AI verification is labor-intensive; tool versions drift.

### PROJECT STATUS & NEXT MILESTONE

Status: <<FILL current increment + evidence>>
Next: <<FILL planned increment / date>>
Open risks: <<FILL>>

### GITHUB QR / REPOSITORY

<<INSERT QR>>
<<FILL repo URL + commit>>

### VIDEO / SHARING

<<INSERT OPTIONAL QR>>
YouTube: <<FILL>> • LinkedIn: <<FILL>>

**TAKEAWAY: AI MAY PROPOSE; ONLY A TEST OR A PRIMARY SOURCE MAY CONFIRM. • Build runnable increments; define Done; record enough for a stranger to reproduce.**

REFERENCES: [1] Course process models [2] BLAST rubric [3] Scrum Guide [5–7] OQS / liboqs /